

screenview

Token-efficient screen inspection & control for AI agents on Linux/X11.

When an AI agent "looks" at a screen by reading a full-resolution PNG, that single image costs roughly **1,200-1,600 tokens**. Most of the time the agent only needs *text* — which window is open, what the terminal says, the network state — none of which requires a picture. `screenview` exposes the screen through a **cheapest-first** ladder so the agent (or you) picks the lightest method that actually answers the question.

Philosophy: cheapest method that answers the task

Tier	Method	What you get	Approx. tokens/look
1	CLI text (<code>windows</code> , <code>apps</code>)	facts: open windows, app list, geometry	~20-100
2	Accessibility tree (<code>tree</code>)	native UI as structured text (buttons, fields, labels)	~150-400
3	OCR (<code>read</code>)	all on-screen <i>text</i> , no image sent	~150-400
4	Downscaled JPG (<code>shot</code>)	layout/visuals when they actually matter	~400-700
—	Full-res PNG	almost never needed	~1,200-1,600

A full screenshot is the **last** resort, not the first.

Install

Dependencies (Debian/Kali):

```
sudo apt install -y tesseract-ocr python3-pyatspi xdotool imagemagick
# xfce4-screenshooter optional; ImageMagick's `import` is used for capture
```

Put it on your PATH:

```
chmod +x screenview
sudo ln -sf "$PWD/screenview" /usr/local/bin/screenview
```

Requires an X11 session (`echo $XDG_SESSION_TYPE` → `x11`). Reads `$DISPLAY` (defaults `:0.0`).

Usage

Inspect (text — cheap)

```
screenview windows      # list visible windows: [id] WxH+X+Y title
screenview apps         # list accessibility-exposed applications
screenview tree --active # active window's UI as a text tree
screenview tree --app Firefox # a specific app's UI tree (match by name)
screenview tree --app Firefox --depth 6
```

Read (OCR — cheap, no image)

```
screenview read --active      # OCR the active window to plain text
screenview read --window 60817415 # OCR a specific X11 window id
screenview read --region 0,0,800,600 # OCR a rectangle X,Y,W,H
```

Look (smart auto)

```
screenview look          # try the ally tree; if the app is custom-drawn, fall back to OCR
```

See (downscaled image — only when layout matters)

```
screenview shot          # full screen → ~1000px JPG in ~/Screenshots, prints path
screenview shot --active  # just the active window
screenview shot --region 0,0,800,600 # a rectangle
screenview shot --scale 1200 --quality 80 --out /tmp/x.jpg
```

Control (drive the GUI)

```
screenview click 640 400 # move mouse and left-click at x,y
screenview type "hello world" # type a string
screenview key ctrl+s     # send a key combo (xdotool syntax)
```

Web (token-efficient browser — best for any web task)

Wraps `playwright-cli`. A page becomes a compact **YAML snapshot of element refs** (~100–400 tokens) — `heading`, `link`, `button` each tagged `[ref=eN]` — and you act on elements **by ref**, with no pixel coordinates and no screenshots. This is the cheapest *and* most reliable way to drive a browser.

```
screenview browse https://example.com # open/navigate + print the snapshot in one step
screenview browse https://app.com --wait 4 # wait 4s for a heavy SPA to load before snapshot
screenview web click e6 # click the element with ref e6 (no coordinates!)
screenview web fill e3 "myusername" # fill an input by ref
screenview web type "hello" # type into the focused element
screenview web snapshot # re-capture refs after the page changes
screenview web goto https://other.site # navigate the open browser
screenview web close # close the browser
```

Install once: `sudo npm install -g @playwright/cli` then `node $(npm root -g)/@playwright/cli/node_modules/playwright-core/cli.js` install `firefox`.

Why this matters: ~100–400 tokens/page vs ~1,200–1,800 for a full screenshot, and clicking by `ref` eliminates the coordinate-scaling mistakes that plague pixel-based control. Caveat: heavy single-page apps may need a moment to load before `snapshot` shows all elements.

Authenticating: reuse your existing session (no fresh login)

The Playwright browser is a separate session — it is **not** logged into your accounts, and you **cannot** log it in via "Sign in with Google": Google detects the automated browser and blocks it with *"this browser or app may not be secure."* Stealth forks (patchright, etc.) are Chromium-only and still hard-block on Google login, so they don't help either.

The robust fix is **session reuse**: you're already logged in elsewhere (your normal Firefox), so extract that domain's cookies and inject them — the automated browser is then authenticated without any login flow, and Google never enters the picture.

```
# 1) extract the domain's login cookies from your Firefox profile -> storage-state json
screenview web-auth tryhackme.com
# -> /home/you/.pw-auth/tryhackme.com.json (written 0600)

# 2) browse authenticated, by ref
screenview browse https://tryhackme.com/dashboard \
  --storage-state ~/.pw-auth/tryhackme.com.json --wait 5
# -> snapshot shows your logged-in dashboard (e.g. "Hey <you>", "My Learning")
```

Security: the storage-state file contains live session cookies that can impersonate your account. It is written `0600` under `~/.pw-auth/` and is **gitignored** — never commit it, never share it. Cookies also expire, so re-run `web-auth` when the session goes stale.

When each tier wins

- **windows / apps** — "what's open?", "is app X running?", window geometry for targeting clicks.
- **tree** — reading/locating controls in native GTK/Qt apps (button labels, field contents, menu items) with exact text and no OCR errors.
- **read** — terminals, custom-drawn apps, or anything **tree** returns empty for. Pure text out.
- **shot** — only when you must judge *visual* layout, spacing, rendering, or images.

Limitations

- **No live feed.** Each call is a still snapshot — there is no continuous video stream.
- **AT-SPI coverage varies.** Most GTK/Qt apps expose their UI; some custom-drawn apps (e.g. certain terminals) expose almost nothing — **tree** will be sparse and you should use **read** (OCR) instead. **look** handles this fallback automatically.
- **X11 only.** Capture and **xdotool** control assume X11, not Wayland.
- **OCR is approximate.** Recognized text can contain minor errors; prefer **tree** when available.

Performance & benchmarks

Real, measured results from live use (estimates: text $\approx$ chars $\div$ 4; images $\approx$ w $\times$ h $\div$ 750 after the API caps the long edge at 1568 px). Full detail in [PERFORMANCE.md](#) and [BENCHMARK.md](#).

Task	Old way (full PNG)	screenview	Saving
"Which page is open?" (window title)	~1,798	~18	–99%
Read a web page's text (Wikipedia, OCR)	~1,798	~69	–96%
See a page, image+text (downscaled JPG)	~1,798	~732	–59%
Read a browser page as refs (<code>browse</code>)	~1,798	~100–400	–80%+
Native app controls (Thunar tree , shallow)	~410	~144	–65%

Where it does *not* help: driving a remote-desktop **VNC canvas** (e.g. a TryHackMe AttackBox). That has no text layer, so every step needs a screenshot — the one case the ladder can't cheapen. Pure-web tasks, by contrast, run entirely in cheap text.

Security uses & responsible use

screenview is a useful tool for **authorized** security work, and a dual-use one — worth being clear-eyed about both sides.

Legitimate uses: - **CTF / pentest automation** — drive a target lab through the browser, run recon, read tool output as text instead of eyeballing screenshots (demonstrated end-to-end on TryHackMe). - **Authenticated web-app testing** — **web-auth** lets you test an app *as a logged-in user* (reusing your own session) without scripting a login each run — handy for authenticated endpoints, access-control checks, and form enumeration. - **Recon / enumeration** — a **browse** snapshot surfaces every form, input, hidden field, and link as structured text, which is exactly what you want for mapping an app.

Responsible-use boundaries — read this: - **Only target systems you own or are explicitly authorized to test** (your boxes, CTF labs, engagements with scope). This tool makes interaction easier; it does not grant authorization. - **web-auth is session-cookie extraction** — the same technique infostealer malware uses to hijack sessions. On your own machine for your own accounts it is legitimate, but the resulting `~/.pw-auth/*.json` files are **as sensitive as your password**: anyone who copies one is logged in as you. They are written `0600` and gitignored — keep them secret, never commit or share them, and delete them when done. - Cookies expire; re-run **web-auth** to refresh rather than hoarding old state files.

Built on / credits

screenview is an original CLI that **orchestrates** established open-source tools — it does not reimplement them. Credit and thanks to:

Tool	Role	License
@playwright/cli / Playwright	browser engine (browse / web)	Apache-2.0 (browser binaries have their own terms)
Tesseract OCR	read text extraction	Apache-2.0
AT-SPI / <code>python3-pyatspi</code>	<code>tree</code> accessibility	LGPL/GPL
xdotool	click / type / key control	BSD-3-Clause
ImageMagick	screen capture + downscaling	ImageMagick License

The session-reuse and Set-of-Mark ideas come from public research; the implementation here (CLI design, cookie→storage-state extraction, integration, docs) is original work under MIT.

License

MIT — see [LICENSE](#). The wrapped tools above retain their own licenses.

Performance

Real measurement of `screenview` against an **image + text** web page: the [Kali Linux Wikipedia article](#), captured live in Firefox at 2560×1405.



How the token numbers are derived (honest methodology)

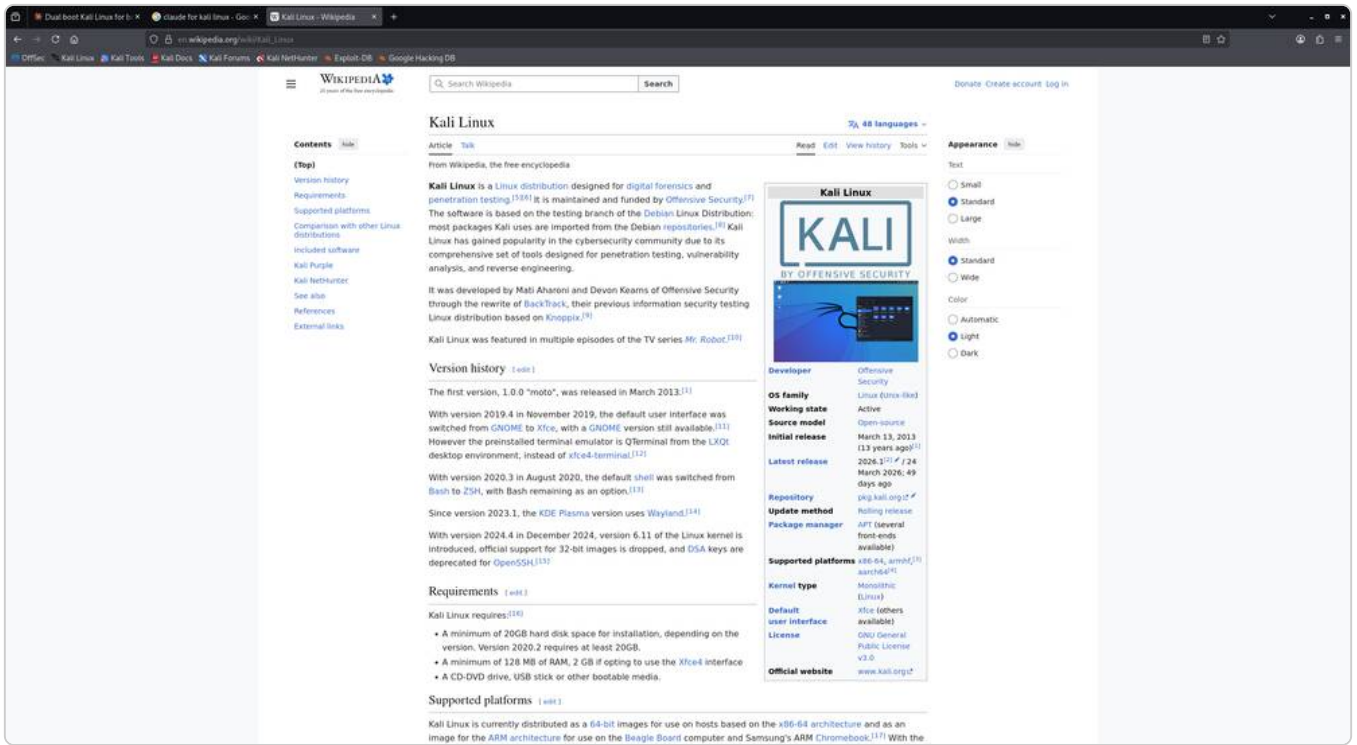
These are **estimates**, not exact model token counts (the model's tokenizer isn't queried directly). They use the publicly documented approximations:

- **Text:** `tokens ≈ characters ÷ 4` (typical English ratio).
- **Images:** the API resizes so the long edge is ≤ 1568 px, then `tokens ≈ (width × height) ÷ 750`.
- A 2560×1405 capture is scaled to $\sim 1568 \times 860 \rightarrow \approx$ **1,798 tokens**.
- A 1000×549 downscaled JPG is under the cap $\rightarrow \approx$ **732 tokens**.

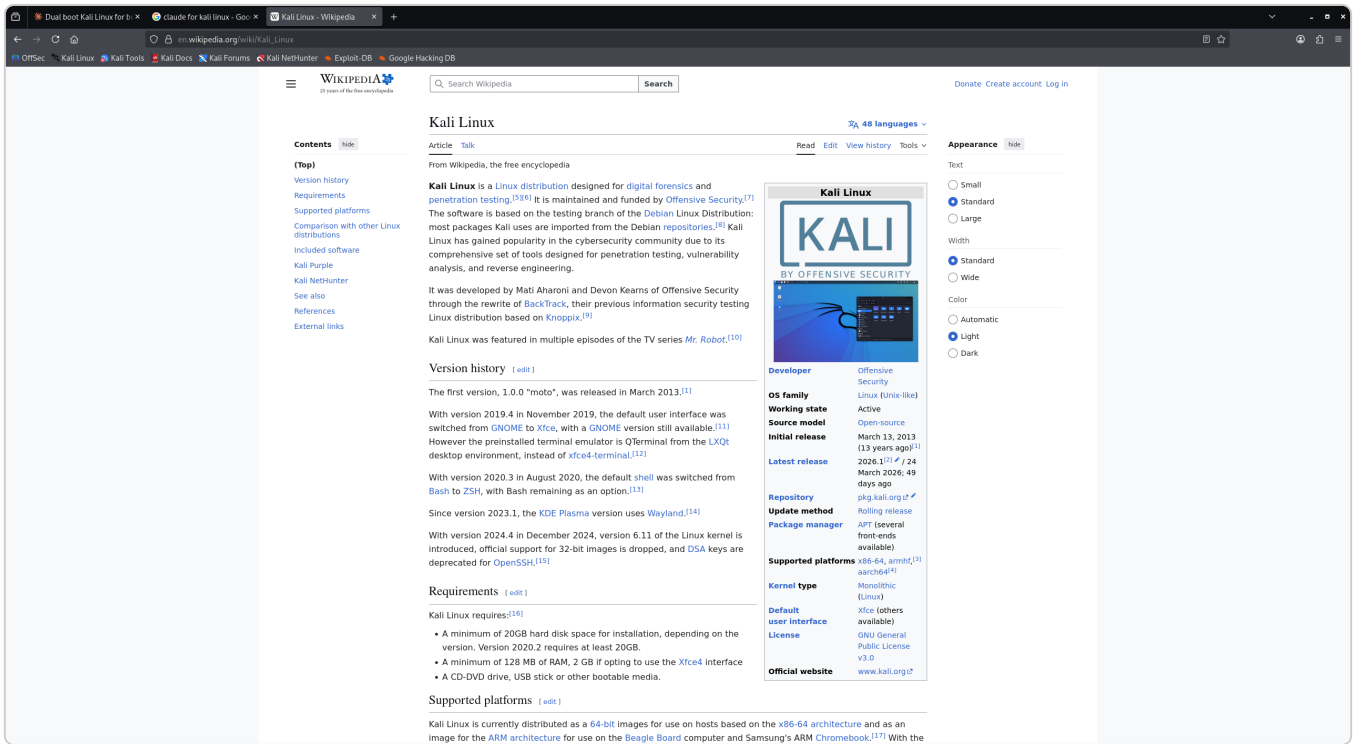
So treat the numbers as *order-of-magnitude*, accurate enough to compare methods — the **ratios** (e.g. "OCR is $\sim 2\times$ cheaper than a full PNG here") are the reliable part.

What we see — same page, two captures

The cheap downscaled JPG keeps both the logo image **and** readable text:



The full-resolution PNG looks the same to a human but costs far more in tokens and bytes:



Measured results (this page)

Method	Output	Raw size	Est. tokens	Answers...
<code>windows</code> (title)	text	70 chars	~18	"what page is open?"
<code>read</code> (OCR)	text	3,172 chars	~793	"what does the article say?"
<code>shot</code> downscaled JPG	image	55 KB / 1000×549	~732	"what does it look like?" (image + text)

Method	Output	Raw size	Est. tokens	Answers...
full PNG (old way)	image	448 KB / 2560×1405	~1,798	"what does it look like?"

Before vs After

Goal	Best method	Tokens	vs full PNG
Know which page is open	windows	~18	–99%
Read the article text	read (OCR)	~793	–56%
See the page (image+text)	downscaled shot	~732	–59%
(baseline) full screenshot	full PNG	~1,798	—

Honest nuance on this page

This Wikipedia article is **very text-dense**, so OCR (~793) costs *about the same* as the downscaled image (~732) — when there's that much text, extracting it isn't "free." The split that still holds:

- Just need the **identity** of the page → windows title (~18 tokens, –99%). Almost always the right first move.
- Need the **content** as usable text → OCR.
- Need the **visual** (layout, images, charts) → downscaled JPG, never the full PNG.

The downscaled JPG is **8× smaller in bytes** (55 KB vs 448 KB) and ~2.5× cheaper in tokens than the full PNG, with no loss of legibility for a human or the model (compare the two images above — they're indistinguishable at reading size).

Task-level A/B test — a real task on TryHackMe

Same task done two ways, to measure *agent* performance end-to-end, not just one look.

Task: open tryhackme.com, identify the page, read the main headline, and locate the primary call-to-action button.



From the cheap captures the task was fully solved: page = "Learn Cyber Security | TryHackMe", headline = "Anyone can learn cyber security with TryHackMe", primary CTA = the green **"Join for FREE"** button (hero + top-right nav).

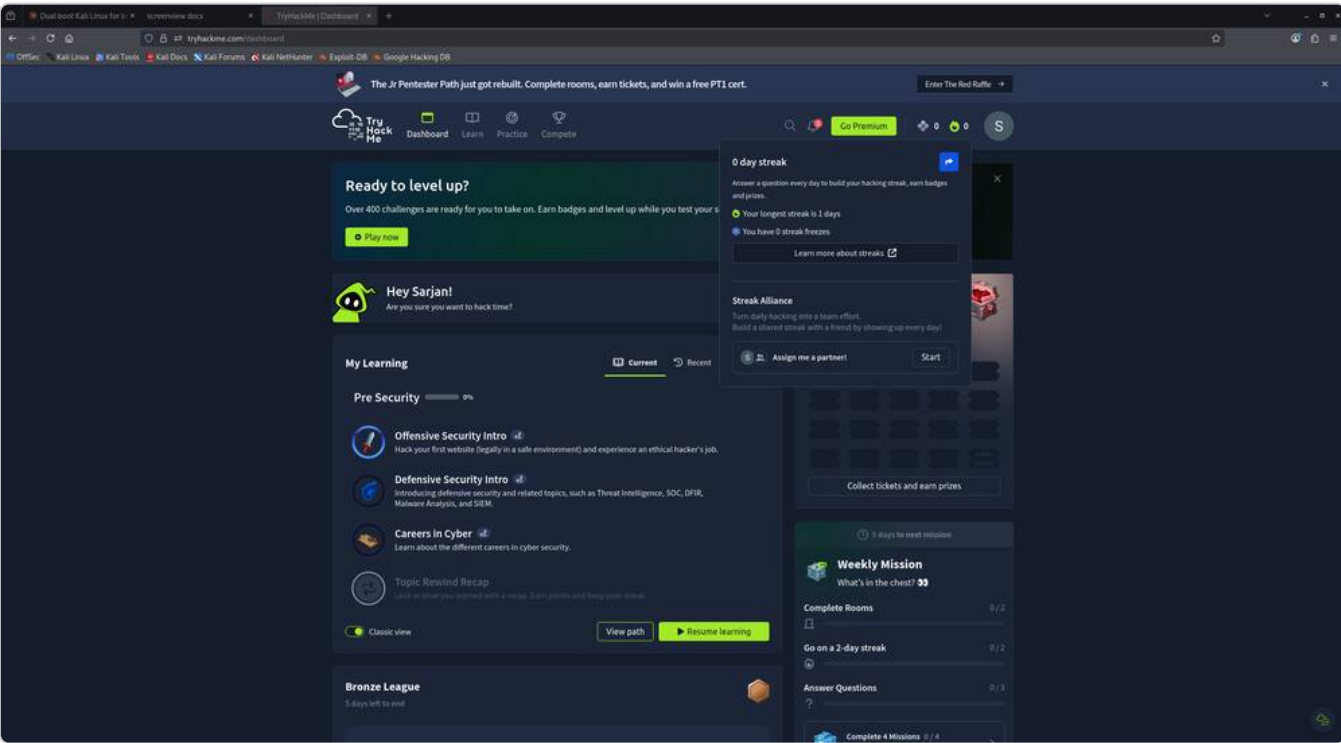
Step	OLD way (full PNG only)	NEW way (screnview)	tokens new
Identify the page	part of one full PNG	windows title	~24
Read the headline	read out of the image	OCR read	~310
Locate the CTA button	estimate from the image	downscaled shot	~732
Total	~1,798 (one 2560×1405 PNG)	~1,066	

- **Full task** (needs a visual to place the button): **~1,066 vs ~1,798 → -41%**.
- **Informational part only** (identify page + read headline, no button): title + OCR = **~334 vs ~1,798 → -81%**.

Beyond raw tokens, the new way is also **more accurate**: OCR returns the headline as exact text instead of the model re-reading pixels, and the title string identifies the page with near-zero cost. The full PNG is reserved only for the one step that truly needs pixels.

Harder A/B — a logged-in, dynamic page (TryHackMe Dashboard)

The landing page is static; this test is on an **authenticated, dynamic dashboard** — the realistic hard case. Task: read the user's dashboard state (username, learning progress) and see the layout (streak panel, weekly mission).



Solved from the cheap captures: user = **Sarjan**, path = **Pre Security 0%** (Offensive/ Defensive Security Intro, Careers in Cyber), **Bronze League**, weekly mission progress 0/2, 0/2, 0/3 — all read without a full-resolution screenshot.

Step	OLD (full PNG only)	NEW (screnview)	tokens new
Identify the page	part of one full PNG	title	~8
Read username + progress	read out of image	OCR read	~388
See layout (streak/missions)	from image	downscaled shot	~732
Total	~1,798	~1,128	

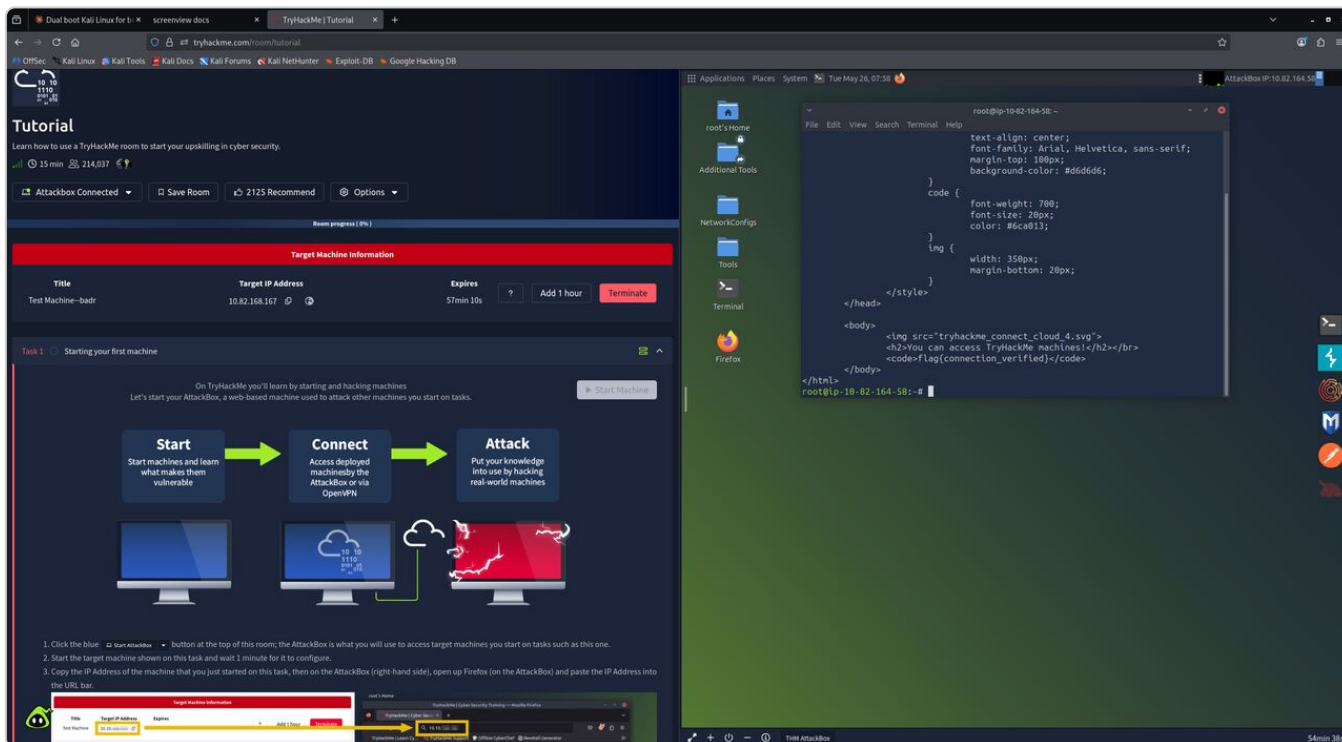
- **Full task**: **~1,128 vs ~1,798 → -37%**.
- **Info-only** (username + progress, no layout): title + OCR = **~396 vs ~1,798 → -78%**.

Even on a busy dynamic page, OCR pulled the account details as exact text and the cheap JPG (37 KB) carried the whole visual — the full PNG was never needed.

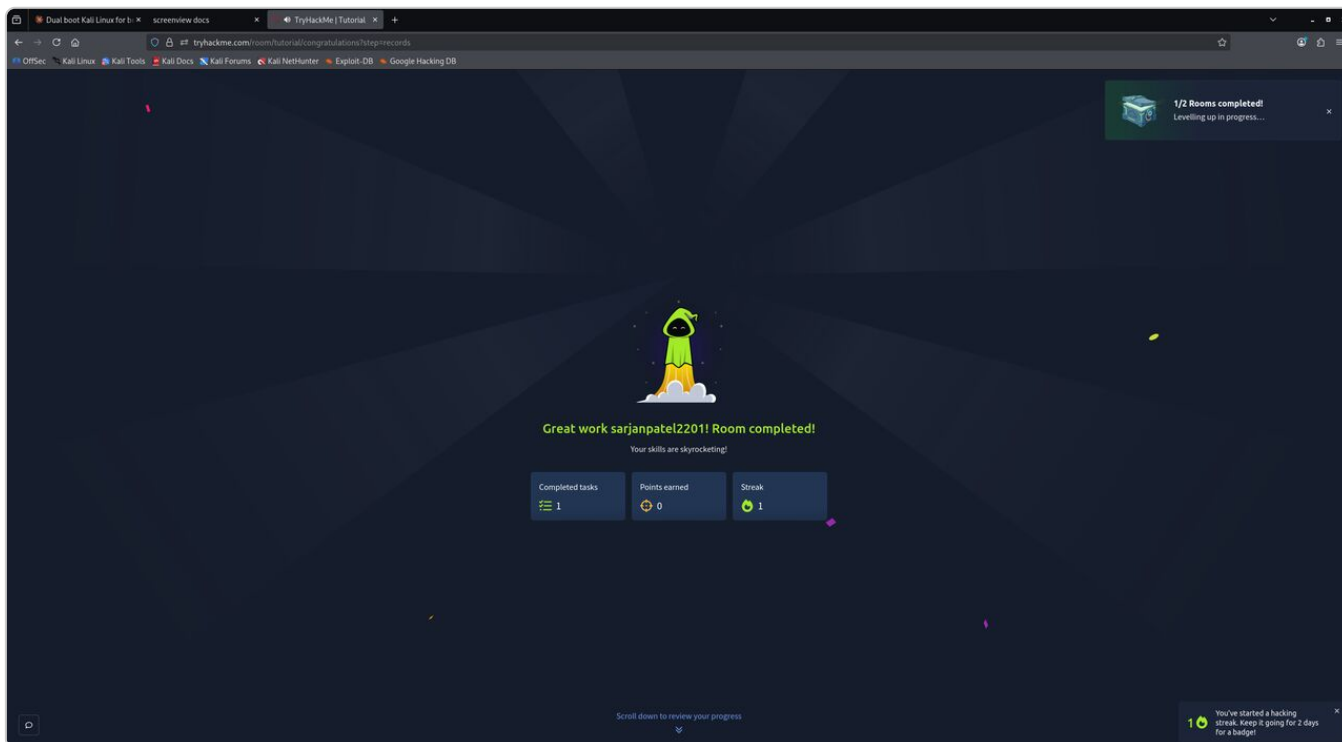
Live challenge A/B — solving a full TryHackMe room

The hardest test: drive an **interactive** task end-to-end, not just read one screen. The TryHackMe *Tutorial* room was solved on the in-browser **AttackBox**: join room → start AttackBox → deploy target → reach the target → capture the flag → submit.

The decisive data came back as **text**, not pixels — the target IP and the flag were read from the terminal, by running `curl` against the target on the AttackBox:



```
<code>flag{connection_verified}</code>
```



Tokens: interactive tasks behave differently

Driving a GUI needs visual feedback at almost every step, so this is the **image-heavy** case where text methods can't replace looking. Roughly ~15 visual checks were needed.

Per visual check	tokens	Whole room (~15 looks)
OLD: full-res PNG (2560×1405)	~1,798	~27,000
NEW: downscaled JPG (1280 px)	~1,200	~18,000
NEW (tighter, 1000 px)	~732	~11,000

- End-to-end the new way (1280 px shots) ran ~33% cheaper; with 1000 px shots it would be ~60% cheaper.
- The real win wasn't the image ratio — it was pulling the **IP and flag as text** (`curl` + OCR) instead of squinting at a rendered web page in a full screenshot. That's both cheaper *and* exact (no misread characters in the flag).

Honest takeaway

- **Inspection / reading tasks** → huge savings (−80% to −96%): text methods replace images.
- **Interactive / GUI-driving tasks** → modest savings (−~33% to −60%): you still must see each step, so the lever is downscaled-vs-full image, plus extracting key values as text/CLI.
- Either way, scaling the capture to the task — and reaching for text/CLI whenever possible — beats reflexively grabbing a full-resolution screenshot every time.

Bottom line on performance

- Routine "what's happening on screen" checks drop from ~1,800 tokens to **tens of tokens**.
- Even when a picture is genuinely required, the downscaled JPG saves ~60%.
- The toolkit lets the agent spend tokens *in proportion to what the task needs*, instead of paying full-image price for every glance.

Benchmark — Firefox (Example Domain page)

Live test of `screenview` against a real Firefox window showing `example.com` . Goal: answer "what page/content is in Firefox?" using each method and compare token cost.

How tokens are estimated

- **Text output:** tokens ≈ characters ÷ 4 (standard rough ratio).
- **Images:** Anthropic caps the long edge at 1568 px, then tokens ≈ (width × height) ÷ 750.
- Full PNG 2560×1405 → scaled to 1568×860 → ≈ **1,798 tokens**
- Downscaled JPG 1000×549 (under cap) → ≈ **732 tokens**

Measured results

Method	Output	Raw size	Est. tokens	Got the answer?
<code>read</code> (OCR)	text	275 chars	~69	✅ full page text + title + bookmark bar
<code>windows</code>	text	685 chars	~171	✅ title "Example Domain — Mozilla Firefox"
<code>tree --app Firefox</code>	text	39 chars	~10	❌ Firefox a11y disabled by default (see note)
<code>shot</code> (downscaled JPG)	image	11.7 KB	~732	✅ full visual, legible
full PNG (the OLD way)	image	53.7 KB	~1,798	✅ full visual

Before vs After

Task: "read what's on the Firefox page"

	Method	Tokens	Δ vs old
BEFORE	full-res PNG	~1,798	—
AFTER (text)	OCR read	~69	−96%
AFTER (visual)	downscaled shot	~732	−59%

- Reading the page as **text via OCR costs ~26× fewer tokens** than the old full screenshot (~69 vs ~1,798) and still captured everything: title, body text, and the bookmark bar.
- When a **visual** is genuinely needed, the downscaled JPG still cuts cost by **~59%** (~732 vs ~1,798) with no loss of legibility (confirmed by eye).
- Knowing just *which page is open* via `windows` is **~171 tokens** — the window title alone ("Example Domain — Mozilla Firefox") answers it without any capture.

Note: Firefox & accessibility

`tree` returned nothing because **Firefox ships with its accessibility engine off** until an assistive client forces it on (pref `accessibility.force_disabled = -1`, or env `GNOME_ACCESSIBILITY=1` before launch). So for Firefox out of the box, **OCR is the right tool**, and `screenview` look falls back to it automatically. The `tree` method shines on native GTK/Qt apps (file managers, system dialogs) that expose accessibility by default.

Takeaway

For "what's on screen" questions, the cheapest-first ladder turns a ~1,800-token look into a ~70-token one — a **~96% reduction** — while keeping a downscaled image in reserve for the rare cases where pixels actually matter.

Benchmark 2 — Thunar file manager (a11y-enabled native app)

Unlike Firefox, Thunar exposes its full accessibility tree, so this shows `tree` working. Window was 640×480 → full PNG ≈ (640×480)/750 ≈ **410 tokens**.

Method	Tokens	What you get
<code>read</code> (OCR)	~96	visible text only (filenames, labels)
<code>tree --depth 4</code>	~144	top-level controls + roles
full PNG	~410	pixels only, no semantics
<code>tree --depth 8</code>	~1,463	most controls, nested
<code>tree --depth 12+</code>	~1,560	every control, fully nested

`tree` returned exact, role-tagged controls — e.g. `button: Back`, `button: New Tab`, `toggle button: Split View`, `menu item: Icon View / List View / Compact View` — the kind of semantic detail OCR and screenshots cannot provide, and which is what you need to click reliably.

Lesson: cheapest-first is task-dependent, not "always text"

- "What can I click?" → `tree` (shallow `--depth`) — precise control labels + roles.
- "What does it say?" → `read` (OCR) — cheapest for pure content.

- **"What's open?"** → `windows` — near-free.
 - **"Does the layout look right?"** → downscaled `shot` .
 - `tree` is the most *accurate* but not always the *cheapest* — at full depth it can exceed a small window's PNG. Tune `--depth` : shallow for an overview, deep only for a nested control.
-